

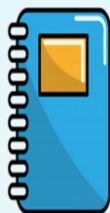
CSE 3203 – Software Engineering

Lecture 07: Agile Methods

Atanu Shuvam Roy



Lecturer,
Dept. of CSE, ULAB, Dhaka



EMAIL:
atanu.shuvam@ulab.edu.bd



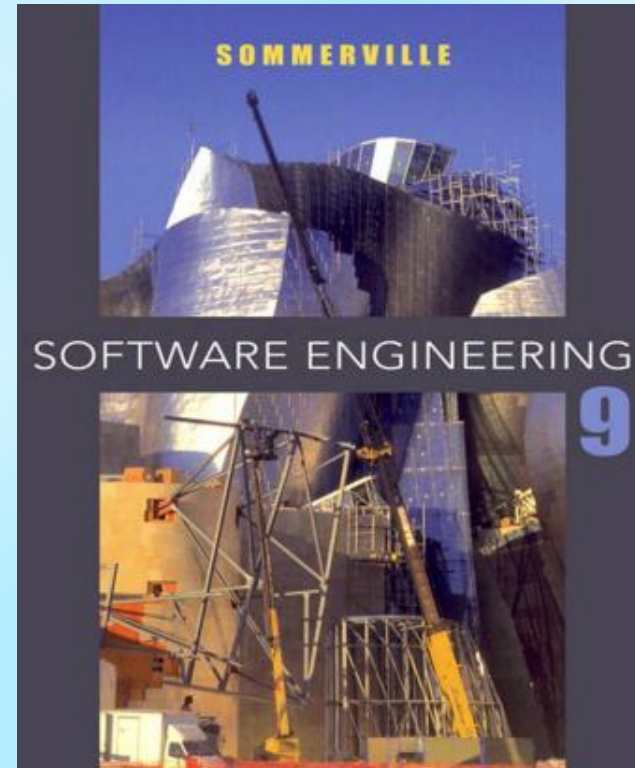
Class Code
byvb74r2



Fall 2025

Book for Reference

- Software Engineering 9th Edition by Ian Sommerville



Mark Distribution



- Mid Semester Exam – 25%
- Final Semester Exam – 40%
- Class Attendance & Performance – 10%
- Course Project – 25%

Is it possible to get A/A+ ?

- Absolutely. Just follow my instructions to the point.

How to fail in this course ?

- Don't follow my instructions.

Instructions



Topics covered

- ✧ Agile methods
- ✧ Plan-driven and agile development
- ✧ Extreme programming
- ✧ Agile project management
- ✧ Scaling agile methods

Rapid software development

- ✧ Rapid development and delivery: the most important requirement for software systems
 - Businesses operate in a fast –changing requirement and impossible to produce a set of stable software requirements
 - Software has to evolve quickly to reflect changing business needs.
- ✧ Rapid software development
 - Specification, design and implementation are inter-leaved
 - System is developed as a series of versions with stakeholders involved in version evaluation
 - User interfaces are often developed using an IDE and graphical toolset.

Agile methods

- ✧ Dissatisfaction with the overheads involved in software design methods of the 1980s and 1990s led to the creation of agile methods. These methods:
 - Focus on the code rather than the design
 - Are based on an iterative approach to software development
 - Are intended to deliver working software quickly and evolve this quickly to meet changing requirements.
- ✧ The aim of agile methods is to reduce overheads in the software process and to be able to respond quickly to changing requirements without excessive rework.

The principles of agile methods

Principle	Description
Customer involvement	Customers should be closely involved throughout the development process. Their role is provide and prioritize new system requirements and to evaluate the iterations of the system.
Incremental delivery	The software is developed in increments with the customer specifying the requirements to be included in each increment.
People not process	The skills of the development team should be recognized and exploited. Team members should be left to develop their own ways of working without prescriptive processes.
Embrace change	Expect the system requirements to change and so design the system to accommodate these changes.
Maintain simplicity	Focus on simplicity in both the software being developed and in the development process. Wherever possible, actively work to eliminate complexity from the system.

Agile method applicability

- ✧ Product development where a software company is developing a small or medium-sized product for sale.
- ✧ Custom system development within an organization: clear commitment from the customer to become involved in the development process, are not a lot of external rules and regulations that affect the software.
- ✧ Because of their focus on small, tightly-integrated teams, there are problems in scaling agile methods to large systems.

Problems with agile methods

- ✧ It can be difficult to keep the interest of customers who are involved in the process.
- ✧ Team members may be unsuited to the intense involvement that characterises agile methods.
- ✧ Prioritising changes can be difficult where there are multiple stakeholders.
- ✧ Maintaining simplicity requires extra work.
- ✧ Contracts may be a problem as with other approaches to iterative development.

Agile methods and software maintenance

- ✧ Most organizations spend more on maintaining existing software than they do on new software development.
- ✧ Two key issues:
 - Are systems that are developed using an agile approach maintainable, given the emphasis in the development process of minimizing formal documentation?
 - Can agile methods be used effectively for evolving a system in response to customer change requests?
- ✧ Problems may arise if original development team cannot be maintained.

Plan-driven and agile development

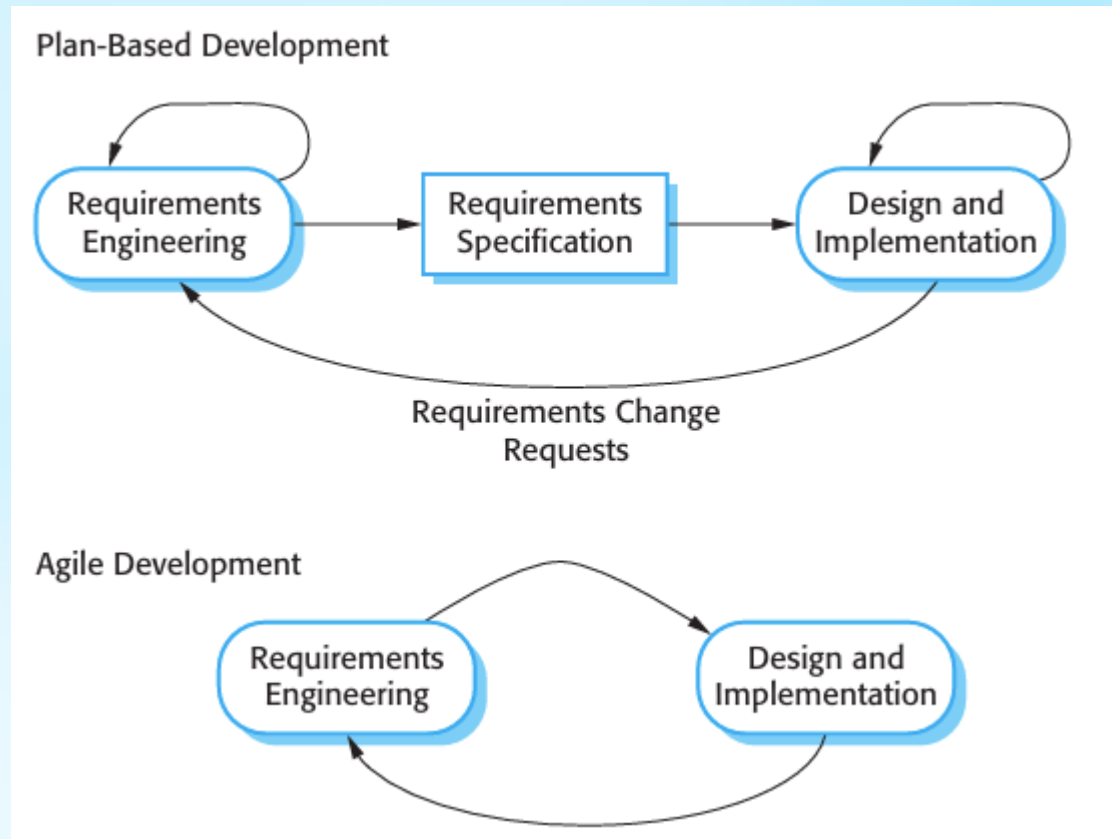
✧ Plan-driven development

- A plan-driven approach: based around separate development stages with the outputs to be produced at each of these stages planned in advance.
- Not necessarily waterfall model – plan-driven, incremental development is possible
- Iteration occurs within activities.

✧ Agile development

- Specification, design, implementation and testing are interleaved and the outputs from the development process are decided through a process of negotiation during the software development process.

Plan-driven and agile specification



Technical, human, organizational issues

- ✧ Most projects include elements of plan-driven and agile processes.
- ✧ Is it important to have a very detailed specification and design before moving to implementation?
- ✧ Is an incremental delivery, where you deliver the software to customers and get rapid feedback from them, realistic?
- ✧ How large is the system that is being developed?

Technical, human, organizational issues

- ✧ What type of system is being developed?
- ✧ What is the expected system lifetime?
- ✧ What technologies are available to support system development?
- ✧ How is the development team organized?

Technical, human, organizational issues

- ✧ Are there cultural or organizational issues that may affect
 - the system development?
- ✧ How good are the designers and programmers in the development team?
- ✧ Is the system subject to external regulation?

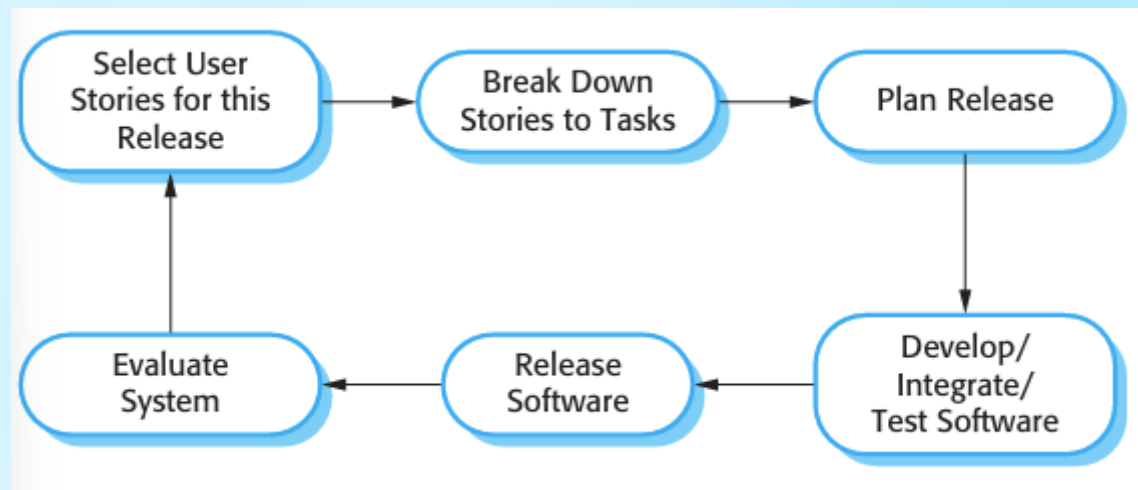
Topics covered

- ✧ Agile methods
- ✧ Plan-driven and agile development
- ✧ Extreme programming
- ✧ Agile project management
- ✧ Scaling agile methods

Extreme programming

- ✧ Perhaps the best-known and most widely used agile method.
- ✧ Extreme Programming (XP) takes an 'extreme' approach to iterative development.
 - E.g. New versions may be built several times per day.

The extreme programming release cycle



Extreme programming practices (a)

Principle or practice	Description
Incremental planning	Requirements are recorded on story cards and the stories to be included in a release are determined by the time available and their relative priority.
Small releases	The minimal useful set of functionality that provides business value is developed first.
Simple design	Enough design is carried out to meet the current requirements.
Test-first development	Tests for a new piece of functionality before that functionality itself is implemented.
Refactoring	All developers are expected to refactor the code continuously as soon as possible code improvements are found.

Extreme programming practices (b)

Pair programming	Developers work in pairs, checking each other's work.
Collective ownership	The pairs of developers work on all areas of the system.
Continuous integration	As soon as the work on a task is complete, it is integrated into the whole system.
Sustainable pace	Large amounts of overtime are not considered acceptable.
On-site customer	A representative of the customer should be available full time for the use of the XP team.

XP and agile principles

- ✧ Incremental development: supported through small, frequent system releases.
- ✧ Customer involvement means full-time customer engagement with the team.
- ✧ People are supported by pair programming, collective ownership and a process that avoids long working hours.
- ✧ Change supported through regular system releases.
- ✧ Maintaining simplicity through constant refactoring of code.

Requirements scenarios

- ✧ In XP, a customer or user is part of the XP team and is responsible for making decisions on requirements.
- ✧ User requirements are expressed as scenarios or user stories.
- ✧ These are written on cards and the development team break them down into implementation tasks. These tasks are the basis of schedule and cost estimates.
- ✧ The customer chooses the stories for inclusion in the next release based on their priorities and the schedule estimates.

A 'prescribing medication' story

Prescribing Medication

Kate is a doctor who wishes to prescribe medication for a patient attending a clinic. The patient record is already displayed on her computer so she clicks on the medication field and can select 'current medication', 'new medication' or 'formulary'.

If she selects 'current medication', the system asks her to check the dose. If she wants to change the dose, she enters the dose and then confirms the prescription.

If she chooses 'new medication', the system assumes that she knows which medication to prescribe. She types the first few letters of the drug name. The system displays a list of possible drugs starting with these letters. She chooses the required medication and the system responds by asking her to check that the medication selected is correct. She enters the dose and then confirms the prescription.

If she chooses 'formulary', the system displays a search box for the approved formulary. She can then search for the drug required. She selects a drug and is asked to check that the medication is correct. She enters the dose and then confirms the prescription.

The system always checks that the dose is within the approved range. If it isn't, Kate is asked to change the dose.

After Kate has confirmed the prescription, it will be displayed for checking. She either clicks 'OK' or 'Change'. If she clicks 'OK', the prescription is recorded on the audit database. If she clicks on 'Change', she reenters the 'Prescribing medication' process.

Examples of task cards for prescribing medication

Task 1: Change Dose of Prescribed Drug

Task 2: Formulary Selection

Task 3: Dose Checking

Dose checking is a safety precaution to check that the doctor has not prescribed a dangerously small or large dose.

Using the formulary ID for the generic drug name, look up the formulary and retrieve the recommended maximum and minimum dose.

Check the prescribed dose against the minimum and maximum. If outside the range, issue an error message saying that the dose is too high or too low.

If within the range, enable the 'Confirm' button.

XP and change

- ✧ Design for change is worth spending time and effort anticipating changes as it reduces costs later in the life cycle.
- ✧ XP, however, maintains that this is not worthwhile as changes cannot be reliably anticipated.
- ✧ Rather, it proposes constant code improvement (refactoring) to make changes easier when they have to be implemented.

Refactoring

- ✧ Programming team look for possible software improvements and implement immediately.
- ✧ This improves the understandability of the software and so reduces the need for documentation.
- ✧ Changes are easier to make because the code is well-structured and clear.
- ✧ However, some changes require architecture refactoring and this is much more expensive.

Examples of refactoring

- ✧ Re-organization of a class hierarchy to remove duplicate code.
- ✧ Tidying up and renaming attributes and methods to make them easier to understand.
- ✧ The replacement of inline code with calls to methods that have been included in a program library.

Testing in XP

- ✧ Testing is central to XP and XP has developed an approach where the program is tested after every change has been made.
- ✧ XP testing features:
 - Test-first development.
 - Incremental test development from scenarios.
 - User involvement in test development and validation.
 - Use of automated testing frameworks.

Test-first development

- ✧ Writing tests before code clarifies the requirements to be implemented.
- ✧ Tests are written as programs rather than data so that they can be executed automatically.
- ✧ All previous and new tests are run automatically when new functionality is added, thus checking that the new functionality has not introduced errors.

Test case description for dose checking

Test 4: Dose Checking

Input:

1. A number in mg representing a single dose of the drug.
2. A number representing the number of single doses per day.

Tests:

1. Test for inputs where the single dose is correct but the frequency is too high.
2. Test for inputs where the single dose is too high and too low.
3. Test for inputs where the single dose $\times$ frequency is too high and too low.
4. Test for inputs where single dose $\times$ frequency is in the permitted range.

Output:

OK or error message indicating that the dose is outside the safe range.

Customer involvement

- ✧ The role of the customer: help develop acceptance tests for the stories to be implemented in the next release.
- ✧ The customer who is part of the team writes tests as development proceeds.
- ✧ However, people adopting the customer role have limited time available and so cannot work full-time with the development team.

Test automation

- ✧ Test automation means that tests are written as executable components before the task is implemented
 - Simulate the submission of input to be tested and check that the result meets the output specification. An automated test framework (e.g. Junit) makes it easy to write executable tests and submit a set of tests for execution.
- ✧ As testing is automated, there is always a set of tests that can be quickly and easily executed

XP testing difficulties

- ✧ Programmers prefer programming to testing and sometimes they take short cuts when writing tests.
- ✧ Some tests can be very difficult to write incrementally.
- ✧ It difficult to judge the completeness of a set of tests.

Pair programming

- ✧ In XP, programmers work in pairs, sitting together to develop code.
- ✧ This helps develop common ownership of code and spreads knowledge across the team.
- ✧ It serves as an informal review process as each line of code is looked at by more than 1 person.
- ✧ It encourages refactoring as the whole team can benefit from this.

Pair programming

- ✧ In pair programming, programmers sit together at the same workstation to develop the software.
- ✧ Pairs are created dynamically so that all team members work with each other during the development process.
- ✧ The sharing of knowledge that happens during pair programming is very important as it reduces the overall risks to a project when team members leave.
- ✧ Pair programming is not necessarily inefficient and there is evidence that a pair working together is more efficient than 2 programmers working separately.

Advantages of pair programming

- ✧ It supports the idea of collective ownership and responsibility for the system.
- ✧ It acts as an informal review process because each line of code is looked at by at least two people.
- ✧ It helps support refactoring, which is a process of software improvement.

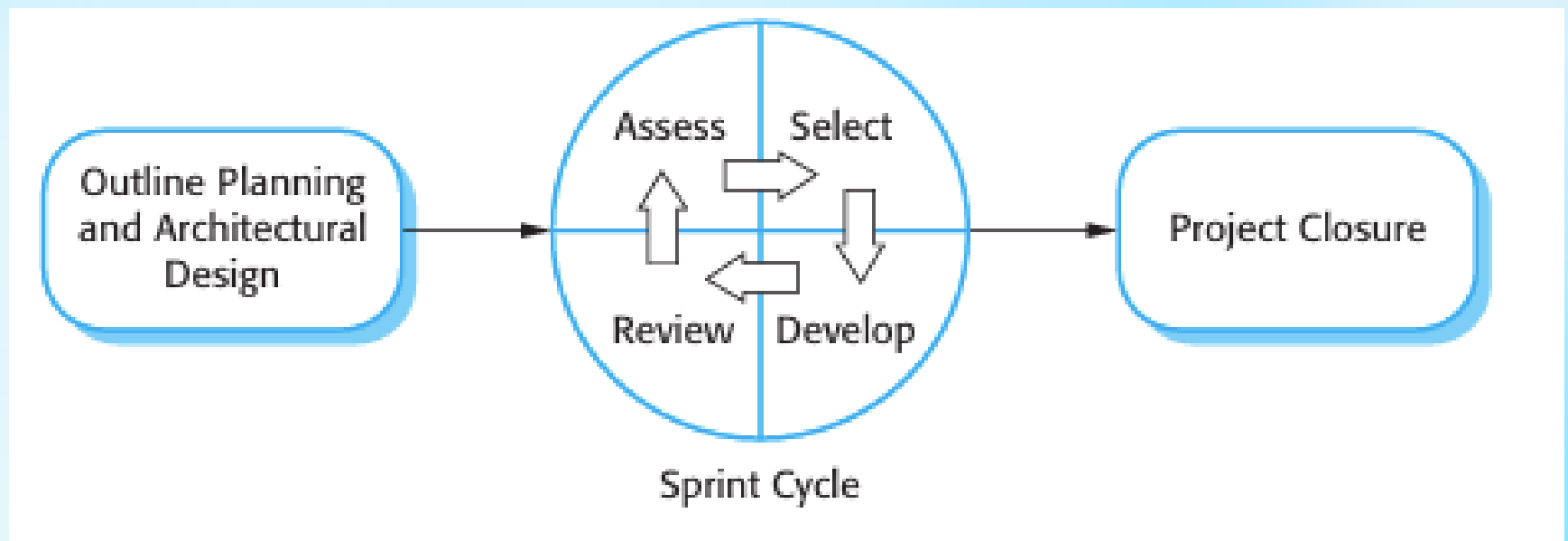
Agile project management

- ✧ Manage the project so that the software is delivered on time and within the planned budget for the project.
- ✧ The standard approach to project management is plan-driven.
- ✧ Agile project management requires a different approach, which is adapted to incremental development and the particular strengths of agile methods.

Scrum

- ✧ The Scrum approach is a general agile method but its focus is on managing iterative development rather than specific agile practices.
- ✧ There are three phases in Scrum.

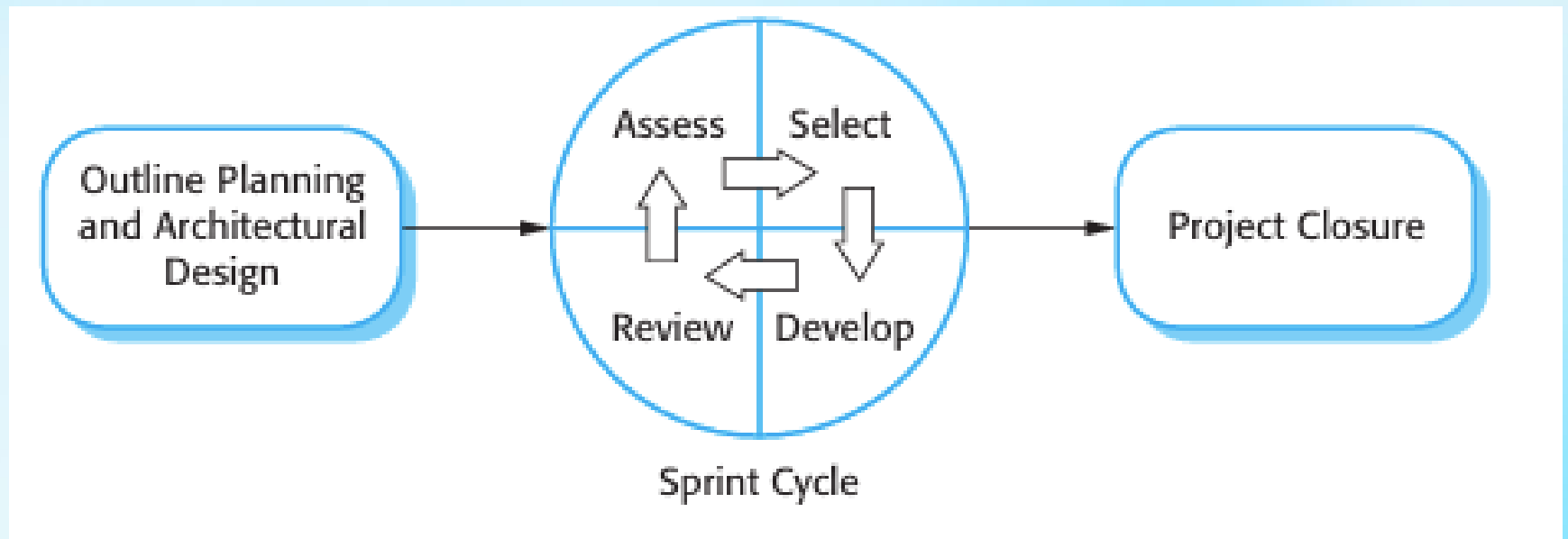
The Scrum process



Scrum

- ✧ There are three phases in Scrum.
 - ✧ The initial phase: outline planning and architectural design.
 - ✧ Sprint cycles.
 - ✧ Project closure.

The Scrum process



The Sprint cycle

- ✧ Sprints are fixed length, normally 2–4 weeks.
- ✧ The starting point for planning is the product backlog, which is the list of work to be done on the project.
- ✧ The selection phase involves all of the project team to select the features and functionality to be developed during the sprint.

The Sprint cycle

- ✧ Once these are agreed, the team organizes themselves to develop the software.
- ✧ The role of the Scrum master is to protect the development team from external distractions.
- ✧ At the end of the sprint, the work done is reviewed and presented to stakeholders. The next sprint cycle then begins.

Teamwork in Scrum

- ✧ The 'Scrum master' is a facilitator who arranges daily meetings, tracks the backlog of work to be done, records decisions, measures progress against the backlog and communicates with customers and management outside of the team.
- ✧ The whole team attends short daily meetings where all team members share information, describe their progress since the last meeting, problems that have arisen and what is planned for the following day.



Thank You

atanu.Shuvam@ulab.edu.bd

University of Liberal Arts Bangladesh
Department of Computer Science & Engineering